ENSAM · FILIÈRE IAGI

RAPPORT DE PROJET – PYTHON AVANCÉ

PickMyDoc

Application de gestion de rendez-vous médicaux

Python · Tkinter · SQLite

RÉALISÉ PAR

Fatima Ezzahra BEN ET-TAHIR

Hanaa MAGHANIM

Yassmine HAMIM

ENCADRÉ PAR

Pr. Mustapha HAIN

École Nationale Supérieure d'Arts et Métiers · ENSAM

Novembre 2026

Table des matières

Introduction générale **3**

Chapitre 1 : Présentation du projet **4**

1.1 Objectifs 4
1.2 Fonctionnalités 4
1.3 Technologies utilisées 4

Chapitre 2 : Analyse et conception **4**

2.1 Besoins fonctionnels 4
2.2 Diagramme de classes 5
2.3 Base de données et relations 5

Chapitre 3 : Réalisation **5**

3.1 Organisation du code 5
3.2 Les classes métier 5
3.3 Accès à la base de données 6
3.4 Éviter les doubles réservations 6
3.5 L'interface Tkinter 6
3.6 Notions de Python avancé mobilisées 7

Chapitre 4 : Interfaces de l'application **7**

4.1 Onglet Patients 7
4.2 Onglet Médecins 8
4.3 Onglet Consultations 8

Conclusion générale **10**

Introduction générale

Prendre rendez-vous chez un médecin n'est pas toujours simple : il faut trouver le bon spécialiste dans sa ville, savoir s'il est disponible, puis retenir la date convenue, la déplacer ou l'annuler si besoin. Une application de gestion permet de rassembler ces informations au même endroit et d'éviter les erreurs, comme deux rendez-vous au même moment pour un même médecin.

La question qui guide ce projet est la suivante : comment concevoir une application simple pour gérer les patients, les médecins et leurs consultations, tout en gardant des données cohérentes ? Pour y répondre, nous avons développé **PickMyDoc**, une application de bureau écrite en Python, qui s'appuie sur une base de données SQLite et une interface graphique Tkinter. Elle respecte les consignes du module : un groupe de trois étudiants, une application de gestion et une base de données limitée à trois tables.

Ce rapport comporte quatre chapitres. Le premier présente le projet, le deuxième son analyse et sa conception, le troisième sa réalisation technique et le dernier les interfaces de l'application.

Chapitre 1 : Présentation du projet

PickMyDoc est une application de bureau qui permet de rechercher des médecins et de gérer des rendez-vous médicaux. Elle repose sur trois notions : le patient, le médecin et la consultation qui les relie.

1.1 Objectifs

L'application doit permettre de :

- rechercher des médecins par nom, spécialité et ville ;
- réserver un rendez-vous sans créer de conflit d'horaire ;
- consulter la liste des rendez-vous ;
- modifier ou annuler un rendez-vous.

1.2 Fonctionnalités

L'utilisateur dispose de trois espaces. Dans le premier, il gère les **patients** (ajout, modification, suppression, avec un email facultatif). Dans le deuxième, il gère les **médecins**, décrits par leur spécialité, leur ville et leur tarif en dirhams, et il peut les rechercher grâce à une barre de recherche. Dans le troisième, il gère les **consultations** : il réserve un créneau, le modifie, l'annule, le marque comme terminé ou affiche les détails. À chaque étape, l'application contrôle la saisie : champs obligatoires, tarif numérique, format de la date et de l'heure, disponibilité du créneau.

1.3 Technologies utilisées

Python porte la logique de l'application et les classes métier. **Tkinter**, avec son module *ttk*, fournit l'interface : onglets, formulaires et tableaux. **SQLite** stocke les données dans un simple fichier, *pickmydoc.db*, sans serveur à installer. Le développement a été fait sous PyCharm et le code source est déposé sur GitHub.

Chapitre 2 : Analyse et conception

2.1 Besoins fonctionnels

L'utilisateur doit pouvoir :

- enregistrer et gérer les patients ;
- enregistrer et gérer les médecins ;
- rechercher un médecin par nom, spécialité et ville ;
- réserver une consultation avec un médecin à une date et une heure données ;
- modifier l'horaire, annuler, terminer ou consulter les détails d'une consultation.

Côté qualité, l'application doit rester **simple** à utiliser, **fiable** grâce à des messages d'erreur clairs, **persistante** (les données survivent à la fermeture) et **cohérente** : un patient ou un médecin déjà lié à une consultation ne peut pas être supprimé.

2.2 Diagramme de classes

Le modèle objet compte trois classes. Un patient et un médecin peuvent avoir plusieurs consultations, tandis qu'une consultation concerne un seul patient et un seul médecin.

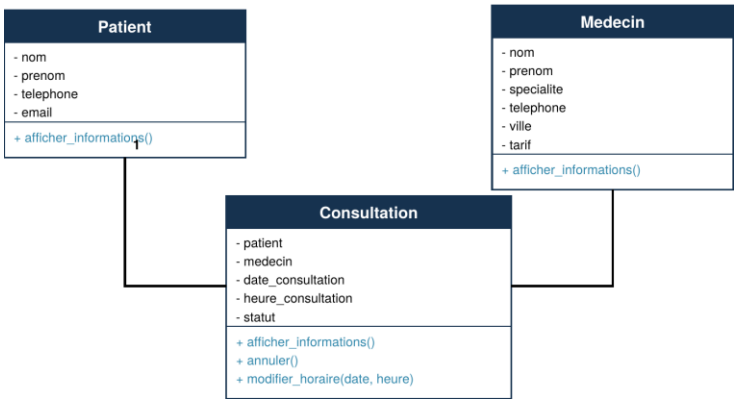


Figure 1 : diagramme de classes de PickMyDoc

2.3 Base de données et relations

La base respecte la limite de trois tables. La table **Patient** contient l'identifiant, le nom, le prénom, le téléphone et un email facultatif. La table **Medecin** contient l'identifiant, le nom, le prénom, la spécialité, le téléphone, la ville et le tarif. La table **Consultation** relie les deux autres : elle porte la date, l'heure et le statut, ainsi que deux clés étrangères, vers le patient et vers le médecin.

Le statut est limité à trois valeurs (*Planifiée*, *Annulée*, *Terminée*) par une contrainte de la base, et vaut « Planifiée » par défaut. Grâce aux clés étrangères, SQLite refuse de supprimer un patient ou un médecin qui a encore des consultations.

Chapitre 3 : Réalisation

3.1 Organisation du code

Le projet sépare clairement trois responsabilités. Le dossier **models** contient les classes *Patient*, *Medecin* et *Consultation*. Le fichier **database.py** crée la base et regroupe toutes les requêtes SQL. Le fichier **gui.py** construit l'interface et lance l'application. Enfin, le fichier **main.py** crée un médecin, un patient et une consultation, les affiche, puis annule la consultation : il sert à tester les classes sans l'interface. Dans l'application, ces mêmes classes sont utilisées par le bouton « Détails ».

3.2 Les classes métier

Chaque classe possède un constructeur et une méthode *afficher_informations()* qui renvoie une description lisible de l'objet. La classe *Consultation* démarre avec le statut « Planifiée » et propose des méthodes pour l'annuler ou changer son horaire.

```python
class Consultation:  2 usages  Hanaa
    def __init__(self, patient, medecin, date_consultation, heure_consultation):
        self.patient = patient
        self.medecin = medecin
        self.date_consultation = date_consultation
        self.heure_consultation = heure_consultation
        self.statut = "Planifiée"

    def afficher_informations(self):  3 usages  Hanaa
        return (
            f"Patient : {self.patient.prenom} {self.patient.nom}\n"
            f"Médecin : Dr {self.medecin.nom} {self.medecin.prenom}\n"
            f"Date : {self.date_consultation}\n"
            f"Heure : {self.heure_consultation}\n"
            f"Statut : {self.statut}"
        )

    def annuler(self):  1 usage  Hanaa
        self.statut = "Annulée"

    def modifier_horaire(self, nouvelle_date, nouvelle_heure):  Hanaa
        self.date_consultation = nouvelle_date
        self.heure_consultation = nouvelle_heure
```

3.3 Accès à la base de données

Toutes les requêtes passent par une même fonction, *executer()*, qui ouvre la connexion, lance la requête, valide la transaction puis referme la connexion. Les valeurs sont transmises sous forme de paramètres, ce qui évite les injections SQL.

```python
def executer(sql, params :Any =(), lire :bool =False):  16 usage
    connexion = sqlite3.connect(DB)
    connexion.execute("PRAGMA foreign_keys = ON")
    try:
        curseur = connexion.execute(sql, params)
        resultat = curseur.fetchall() if lire else None
        connexion.commit()
        return resultat
    finally:
        connexion.close()
```

3.4 Éviter les doubles réservations

Avant toute réservation ou modification d'horaire, la fonction *creneau_disponible()* vérifie que le médecin n'a pas déjà une consultation planifiée au même moment. Lors d'une modification, la consultation en cours est ignorée pour ne pas entrer en conflit avec elle-même.

```python
def creneau_disponible(id_medecin, date, heure, ignorer :int =-1):  2 usages  Windows
    n = executer( sql: "SELECT COUNT(*) FROM Consultation "
                  "WHERE id_medecin=? AND date_consultation=? AND heure_consultation=? "
                  "AND statut='Planifiée' AND id_consultation != ?",
                  params: (id_medecin, date, heure, ignorer), lire=True)[0][0]
    return n == 0
```

3.5 L'interface Tkinter

Pour ne pas répéter le même code, nous avons écrit une page générique, *PageCRUD*, composée d'un formulaire, des boutons Ajouter, Modifier, Supprimer et Vider, et d'un tableau. Elle valide les champs, remplit le formulaire quand on sélectionne une ligne et demande une confirmation avant une suppression. Elle sert directement pour les patients. La page des médecins, *PageMedecins*, en hérite et

ajoute la barre de recherche ; la page des consultations, *PageConsultations*, gère les réservations. La classe *App* assemble le tout dans un *Notebook* à trois onglets.

```
class PageMedecins(PageCRUD):

def rechercher(self):  2 usages  Windows
    self.filtres = {cle: e.get().strip() for cle, e in self.rech.items()}
    self.charger()
    if not self.lignes:
        messagebox.showinfo( title: "Recherche",  message: "Aucun médecin ne correspond à ta recherche.")
```

3.6 Notions de Python avancé mobilisées

Ce projet fait appel à la programmation orientée objet et à l'héritage (*PageMedecins* hérite de *PageCRUD*, *App* de *tk.Tk*), au passage de fonctions en paramètre (la page générique reçoit les fonctions de la base à appeler), à la gestion des exceptions (*ValueError* pour les nombres et les dates, *sqlite3.IntegrityError* pour les suppressions impossibles), au module *datetime* pour valider les formats, et à SQLite avec des requêtes paramétrées, des jointures et des clés étrangères.

Chapitre 4 : Interfaces de l'application

La fenêtre principale, intitulée « PickMyDoc – Gestion des consultations », contient trois onglets.

4.1 Onglet Patients

Un formulaire (nom, prénom, téléphone, email facultatif) et un tableau gèrent les patients. Sélectionner une ligne remplit le formulaire, ce qui permet de modifier ou supprimer. La Figure 2 montre deux patients enregistrés.

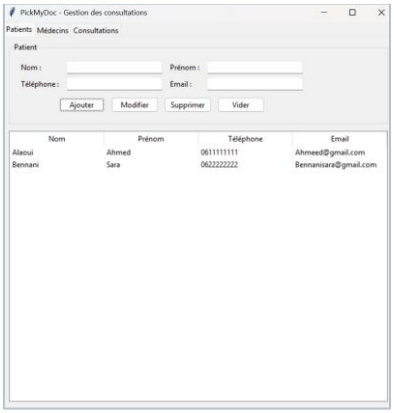


Figure 2 : onglet Patients

4.2 Onglet Médecins

Même fonctionnement, avec en plus une barre « Rechercher un médecin » par nom, spécialité et ville. Dans la Figure 3, la recherche « cardiologie » + « Rabat » ne laisse que le Dr Idrissi Omar parmi les trois médecins enregistrés ; le bouton Réinitialiser réaffiche toute la liste.

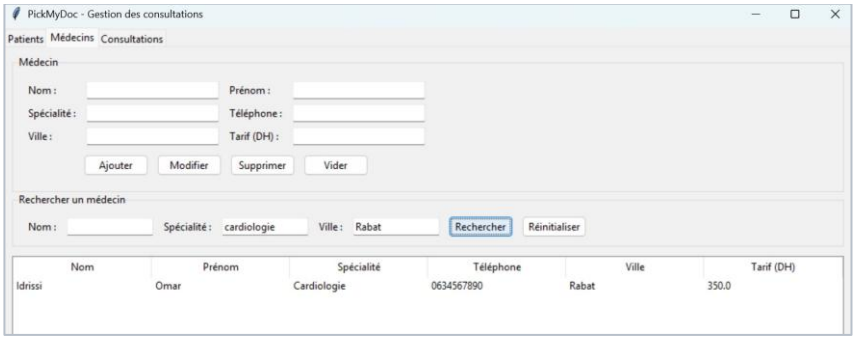


Figure 3 : recherche d'un médecin par spécialité et par ville

4.3 Onglet Consultations

On choisit un patient et un médecin dans deux listes, on saisit la date et l'heure, puis on réserve, modifie, annule ou termine la consultation. Dans la Figure 4, Bennani Sara réserve le Dr Idrissi Omar le 2026-11-10 à 10:30 : la consultation apparaît avec le statut « Planifiée ».

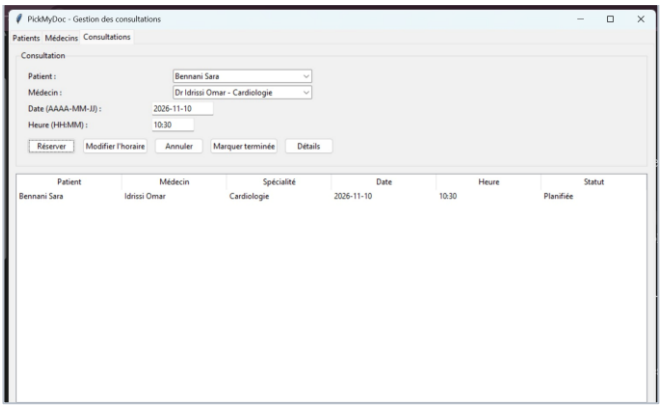


Figure 4 : réservation d'une consultation

Le bouton Détails (Figure 5) rassemble les informations du patient, du médecin et de la consultation.

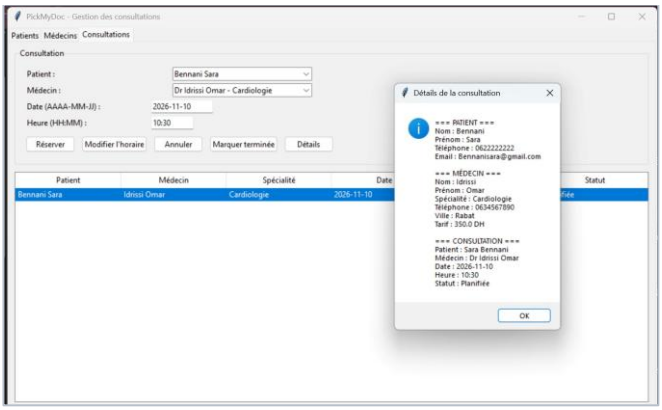


Figure 5 : détails d'une consultation

Si on réserve le même médecin au même créneau pour un autre patient, l'application refuse (Figure 6). C'est le contrôle décrit dans la section 3.4, qui empêche les doubles réservations.

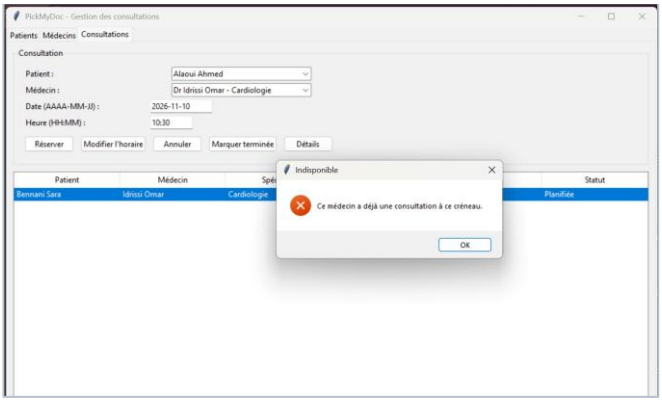


Figure 6 : refus d'une réservation sur un créneau déjà pris

Après un clic sur Modifier l'horaire, la consultation est déplacée au 2026-12-01 à 9:30 et reste « Planifiée » (Figure 7).

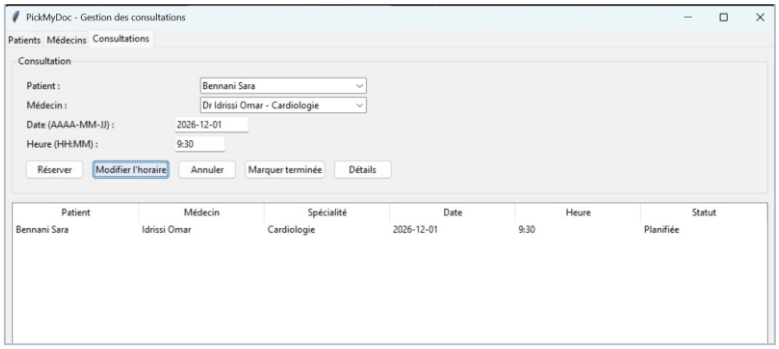


Figure 7 : modification de l'horaire d'une consultation

Avant d'annuler un rendez-vous, l'application demande une confirmation (Figure 8) ; si on répond Oui, le statut passe à « Annulée ».

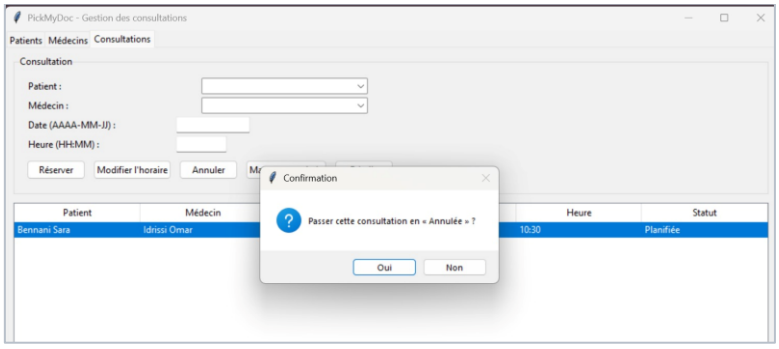


Figure 8 : confirmation avant l'annulation d'une consultation

Conclusion générale

Ce projet nous a permis de mettre en pratique ce que nous avons appris dans le module Python avancé : la programmation orientée objet, la manipulation d'une base de données SQLite et la création d'une interface graphique avec Tkinter. PickMyDoc permet de gérer les patients, de rechercher des médecins et de planifier, modifier ou annuler des consultations, avec des contrôles qui gardent les données cohérentes.

Nous avons surtout dû résoudre quatre difficultés : empêcher deux consultations au même créneau pour un même médecin, valider correctement les saisies (champs vides, nombres, dates et heures), relier la partie objet à la base de données, et nous répartir le travail au sein du groupe.

Code source : https://github.com/maghanimhanaa-hub/PickMyDoc